cs-by-hand-format

Anna Arpaci-Dusseau

April 2026

1 q2.py Binary Representations

As humans, we are used to counting in base 10, we have 10 distinct digits: 0,1,2,3,4,5,6,7,8,9. When we parse a number, 1,431 for example, we understand this value as being

$$1 * 1000 + 4 * 100 + 3 * 10 + 1$$

. Effectively, each digit is a *weight* applied to a power of 10.

Computers, naturally, however only have access to 2 distinct values LOW (voltage) and HIGH. Thus, it is natural to represent values in **base 2** when we are talking about data in computing systems.

To work with **base 2** or **binary**, the basic same idea of weights and powers applies. However, now when we see a value 11011, we should understand this to mean 1*2^4 + 1*2^3 + 0*2^2 + 1*2^1 + 1 *2^0.

To represent a decimal value in binary, work from left to right. Take out the largest possible power of two from the decimal value, write a 1 in the corresponding digit, and continue to work until you reach the "one's place".

Write 4 in binary

2^3	2^2	2^1	2^0

Write 28 in binary

2^5	2^4	2^3	2^2	2^1	2^0

2 q1.py - 2's Complement Circle

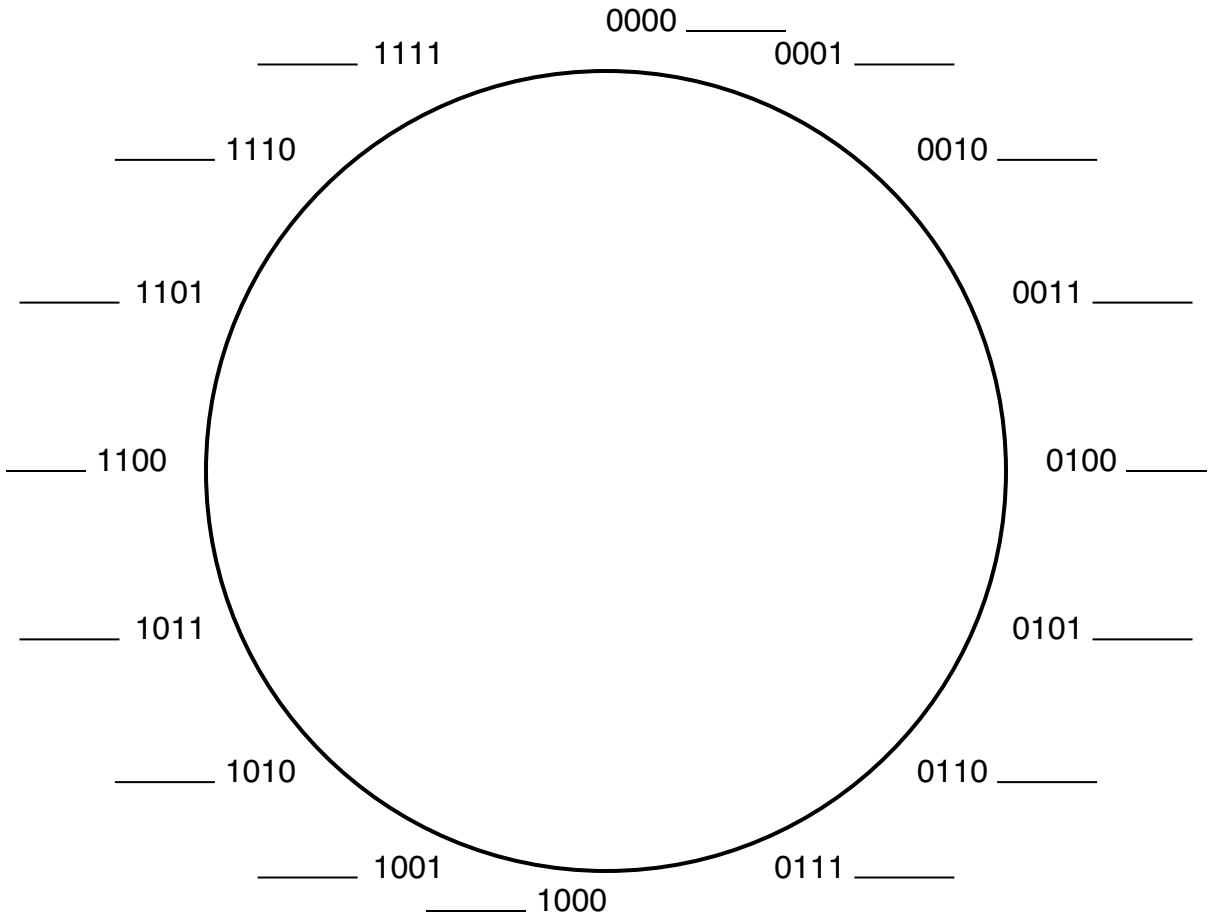
Now, what about negative values?

The most common convention to represent **signed integers** in binary is 2's complement.

To represent a negative value in 2's complement: 1. Write the positive version of the number in binary 2. Flip each digit (0->1 or 1->0) 3. Add 1 in binary (remember to carry if needed!)

Doing the procedure again (starting with a negative version of the number in binary) will flip a negative value to a positive. Notice, that positive values will always have a zero as the highest order bit, while negative values will have a 1 in the highest order bit. As such, we call the highest order bit, the **sign bit**. Consider: To represent the value positive 4 in 2's complement, you will need 4 bits: 0100.

Fill in the circle with the decimal value of the following 4-bit 2's complement binary around the circle.



Consider drawing arrows on the circle representing:

1. Add 4 to 1.
2. Add 2 to -1.
3. Subtract 3 from 2.
4. Subtract 3 from -1.

Notice how adding always goes clockwise around the circle and subtracting goes counter-clockwise. 2's complement representation makes binary arithmetic continuos, with no special cases for negatives, positives, etc.

Except– consider what happens when we add 3 to 6...

This would result in **overflow** as we can not represent 9 in 4-bit 2's complement. Instead, the operation leaves us with a representation of a negative value in 4-bit 2's complement.

3 q2.py Binary Representations with Negatives

Write the following signed integer in 2's complement binary.

Write -4 in binary

A[3]	A[2]	A[1]	A[0]

Write -17 in binary

A[5]	A[4]	A[3]	A[2]	A[1]	A[0]

4 q3.py Binary Arithmetic

Consider we can also do arithmetic with binary values just as we can with decimal values.

However, now when we do 1+1 in a single column, we must carry a 1 (from the resulting 10), to the next column. (This is similar to how we would carry a "1" to the tens place from adding 5+5 in decimal.)

Represent -7+-6

Cin[3]	Cin[2]	Cin[1]	Cin[0]
			0

	A[3]	A[2]	A[1]	A[0]
+	B[3]	B[2]	B[1]	B[0]
	S[3]	S[2]	S[1]	S[0]

Represent -5+6

Cin[3]	Cin[2]	Cin[1]	Cin[0]
			0

	A[3]	A[2]	A[1]	A[0]

+	B[3]	B[2]	B[1]	B[0]

S[3]	S[2]	S[1]	S[0]

5 q4.py Hex/Binary Table

Now, in computer systems it can become unwieldy and error-prone to transcribe long binary values by hand! Furthermore, we have seen that translating from decimal->binary or vice versa requires a bit of math.

However, let's now consider **base 16** or **hexidecimal**. Why introduce a new base?

1. Base 16, like base 10, has enough digits to be compact and easy for humans to read and write.
2. Base 16 has a nice property– four binary digits correspond to exactly one hex digit. Thus, converting a binary value to hex is as simple as grouping digits into fours, and then a one-to-one translation!

If you understand base 10 as the base for humans, and base 2 as the base for computers, base 16 is the base for programmers (the middle man so to speak)!

Complete the table below.

Complete the Table

Dec	Bin	Hex
		0x0
		0x1
		0x2
		0x3
		0x4
		0x5
		0x6
		0x7
		0x8
		0x9
		0xa
		0xb
		0xc
		0xd
		0xe
		0xf

6 q5.py Hex Representation

Complete the following binary->hex conversions or hex->binary conversions. This should be far easier than working with binary->decimal for example!

Hint: Feel free to use the table you completed above.

Fill in the Binary

A[7]	A[6]	A[5]	A[4]	A[3]	A[2]	A[1]	A[0]
0xb				0x0			

Fill in the Hexidecimal

A[7]	A[6]	A[5]	A[4]	A[3]	A[2]	A[1]	A[0]
1	0	1	1	0	0	1	0

Fill in the Binary

A[15]	A[14]	A[13]	A[12]	A[11]	A[10]	A[9]	A[8]	A[7]	A[6]	A[5]	A[4]	A[3]	A[2]	A[1]	A[0]
0x2				0x0				0xb				0x1			

Fill in the Hexidecimal

A[15]	A[14]	A[13]	A[12]	A[11]	A[10]	A[9]	A[8]	A[7]	A[6]	A[5]	A[4]	A[3]	A[2]	A[1]	A[0]
0	0	0	0	0	0	1	1	1	1	0	1	0	0	0	0

6.1 q1.py SOP Expression

6.2 q2.py Logic Gates and Truth Tables

A fundamental computer of computer circuitry is the logic gate. Logic gates take in two signals (HIGH/LOW voltages) and output a HIGH or LOW signal in turn.

In this section, we will see how we can combine this gates to do complicated functions, like adding, storage, etc, and build up to a complete CPU (computer brain!).

Below, we have:

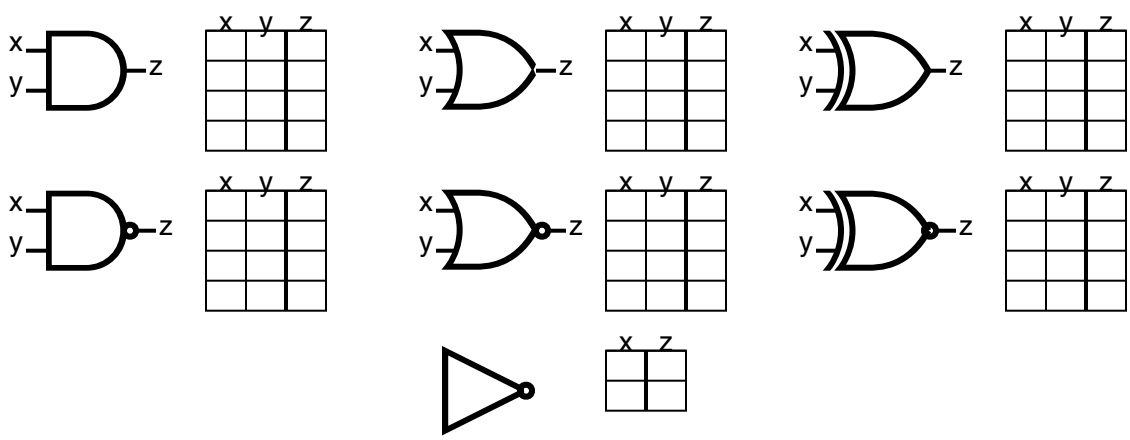
1. AND (produces a 1 if A and B are 1. Otherwise 0)
2. OR (produces a 1 if either A and B, or both, are 1. Otherwise 0.)
3. XOR (produces a 1 if either, but not both, A and B are 1. Otherwise 0.)
4. NAND (not and) (produces a 1 if A and B are 0. Otherwise 1)
5. NOR (not or) (produces a 0 if either A and B, or both, are 1. Otherwise 1.)
6. NOT XOR (produces a 0 if either, but not both, A and B are 1. Otherwise 1.)

The final gate takes is one input, and produces one output.

7. Inverter (not) (if A is 1, produces 0. Otherwise 1.)

We can represent logical statements as truth tables.

On the left side of the bolded truth table line, fill in all the possible input values for A and B. - done effectively, this will look like counting in binary: A=0,B=0 (00), A=0,B=1 (01), A=1... On the right side of the bolded line, fill in the output given the corresponding A and B values.



6.3 q3.py Truth Table, SOP, Draw the circuit

Now, any logic table can be expressed as a "sum of products expression (SOP)". In turn, any SOP expression can be represented in circuitry! Thus, any truth table can be turned into a circuit.

For a given truth table, inspect each row. Say, that row 1, row 3, and row 4 output TRUE (1).

Effectively, the entire truth table can be described as:

output = (condition for row 1) OR (condition for row 3) OR (condition for row 4)

Then, each condition is simply the AND of each variable. For example if we have A=1,B=0,C=1, we can describe that condition as

$$A\bar{B}C$$

Note, that OR is often represented by addition: $+$, AND is represented via multiplication $\cdot$, and NOT is often represented by a bar $\bar{A}$.

A	B	C	X
0	0	0	1
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0

Write a SOP expression for the above truth table.

Draw a circuit that encodes the above SOP expression.

6.4 q4.py Timing Delays

Now, circuits are real-world components. Like all real-world processes, they take time to run and compute!

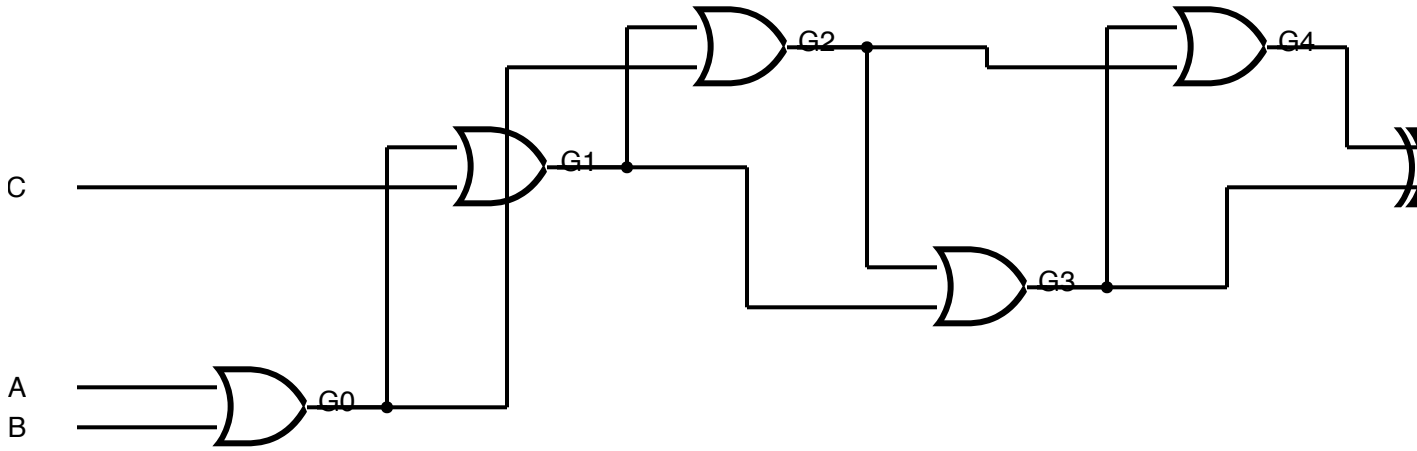
How long? Each circuit module has a timing specification described in two parts:

Tpd is the **propagation delay** which means the time it takes from giving input to getting correct output.

Tcd is the **contamination delay** which means the time it takes from giving input to getting some incorrect (contaminated) reading.

Effectively, when building a circuit and combining modules, after giving new input, after the contamination delay elapses we do not want to read from the output until the propagation delay has passed.

1. To calculate Tpd, find the longest path of summed Tpds of each component, along the path form input->output.
2. To calculate Tcd, find the shortest path of summed Tcds of each component, along the path form input->output.



Gate	Tpd	Tcd
and	34	4
or	41	14
xor	34	7
nand	43	13
nor	43	11
notxor	47	4

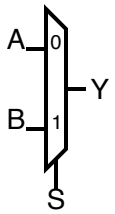
What is the Tpd and Tcd of the above circuit?

6.5 q5.py Control circuits– MUX Truth Table, Implementation

Now, a common control component that will be useful for us is known as the "multiplexer".

A 2-way multiplexer takes in two inputs, and uses a third input, a "select bit" to indicate if we want the output to be the 0th input (S=0) or the 1st input (1st, S=1).

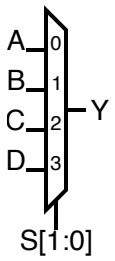
A n-way multiplexer takes in N inputs, and `ceil(log_2(N))` select bits to describe which of the N inputs to chose from.



A	B	S	Y

Write a SOP expression(s) for the truth table.

Draw a circuit that encodes the SOP expression.



S[1]	S[0]	Y

How many select bits do you need for an 12-input MUX?

6.6 q6.py Full Adder

Now, remember from the last section how we did binary arithmetic. Can we represent that operation in circuitry?

First, we must break down and understand the operation between three 1-bit values, A and B and the carry-in bit (Cin). What are the possible outputs for S (the output value) and Cout (the carry out bit).

Just as before, we can represent this scenario as a truth table, then an SOP expression, and then a circuit.

Cin	A	B	S	Cout

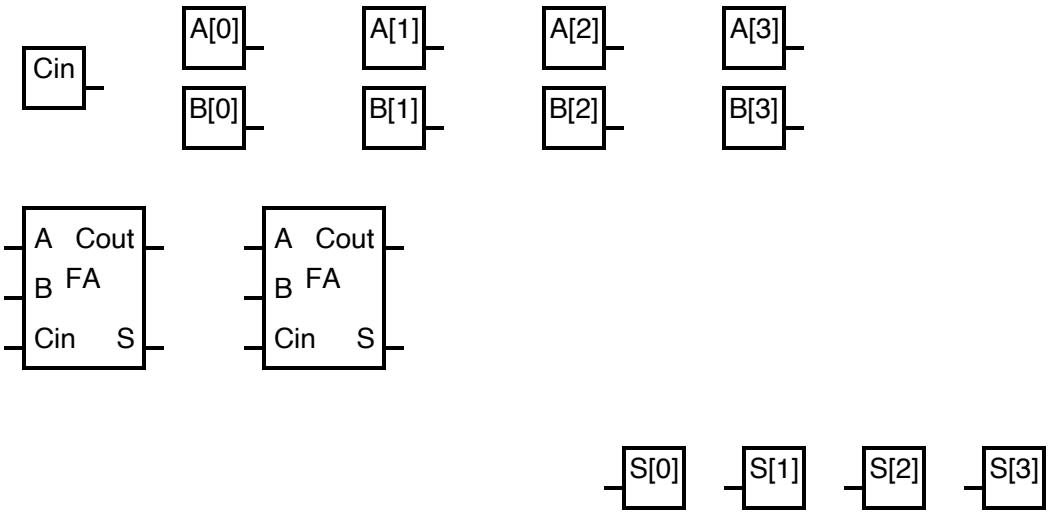
Write a SOP expression(s) for the truth table.

Draw a circuit that encodes the SOP expression.

6.7 q7.py 4-bit full adder

Now, consider, can be combine this full-adder module into a 4-bit adder? To do so, we will build the conceptually simple (and slightly inefficient) **ripple carry adder**. This adder will work by first doing a 1-bit addition on A[0] and B[0], producing S[0] and then a carry out bit. This will be passed along to the next full adder module to calculate A[1],B[1] and so on and so forth.

Complete the 4-Bit Ripple Carry Adder by adding FA modules and wires



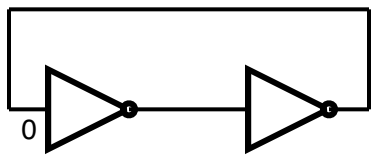
6.8 q1.py Sequential Logic

Now, we must consider how can we "remember" or store information in our circuitry. Computation is great! But to truly make a computer, we must also have **stateful circuits** or circuits with "memory"!

In order to represent stateful circuits, we will create circuits with feedback loops.

Inspect the circuit below.

Consider the labeled signal 0. What is the value on all other wires in the circuit?



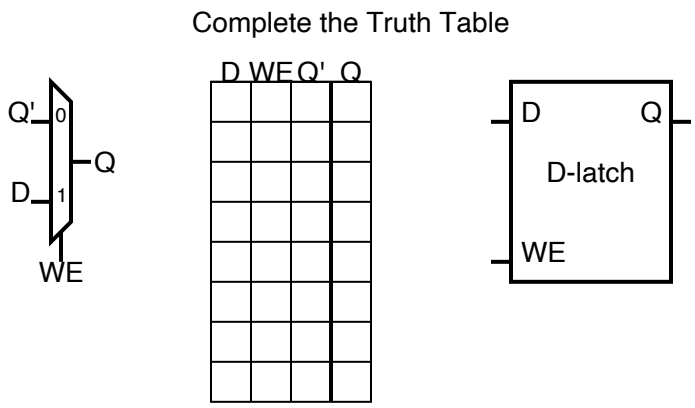
Building off such "feedback" loops we can create stable storage.

6.9 q2.py D-Latch Truth Table

Create a feedback loop, draw a wire connecting Q to Q' in the schematic below. This is a D-Latch.

Note, that we have labeled the "select bit" of the mux "WE". "WE" stands for write-enable. Does this make sense?

Now, fill out the truth table accordingly.



We will represent the mux with feedback loop as the module "D-Latch" with inputs D and WE, and output Q.

1. Which signal is the input data to write to the storage?
2. Which signal is the stored data value?

6.10 q3.py Flip Flop - Draw the Waveform

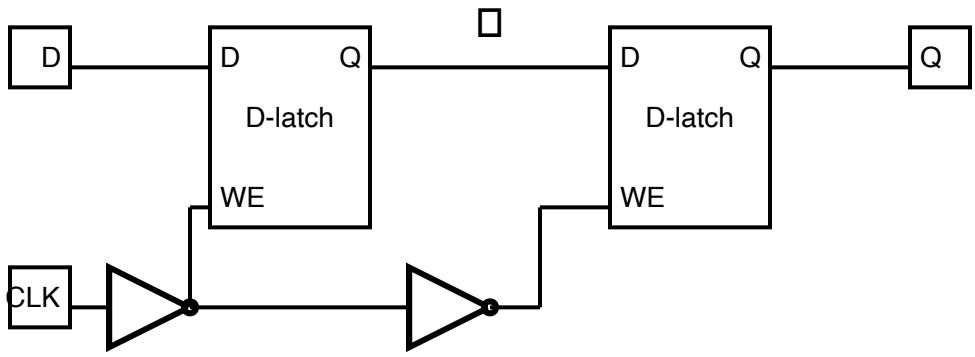
The next question is a bit tricky. You may have noticed from the previous sections that circuits are timing dependent! We must carefully consider, how long does some particular output take to calculate.

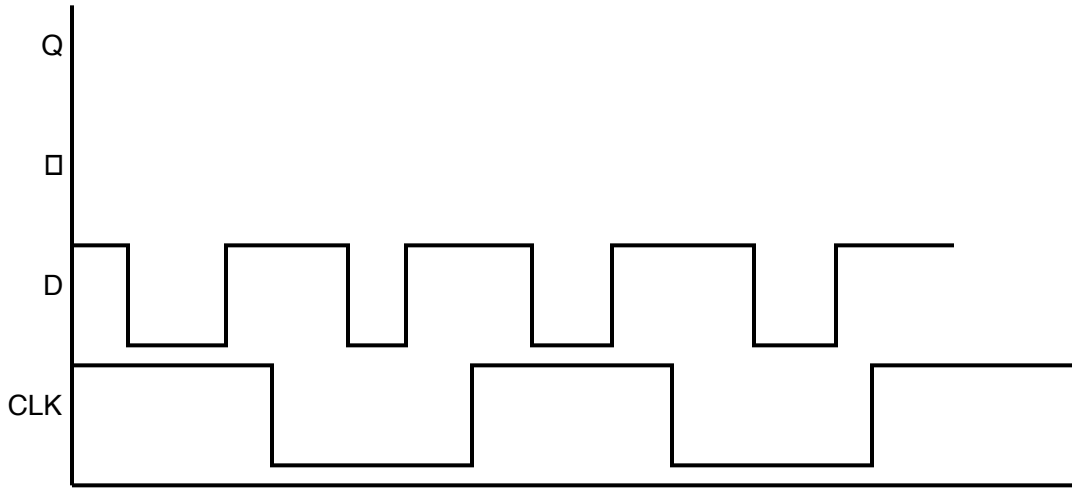
In turn, enabling data to be stored, and then knowing when to read said data is also a question of timing. When is a d-latch taking in new inputs? Anytime WE is enabled! When is a d-latch emitting a new stable, output? After WE is flipped, the duration of the tpd of a mux.

Building circuits around this timing consideration can be troublesome. Instead, we find in practice, it is better to have **clocked storage**. Modern circuits have access to an oscillating clock signal. What if we could take in new inputs on the rising edge of a clock instead? Then, we can read stable values from our storage at any clock high?

Investigate the implementation of the flip flop below and complete the missing waveforms given the CLK and input data.

Draw the signal waveform for Q and □

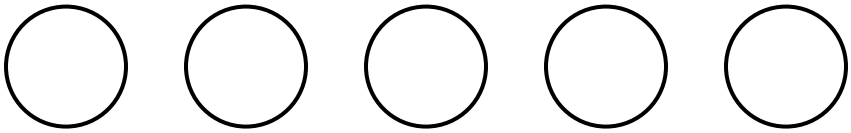




6.11 q4.py FSM – Binary Combination Lock

Complete the State Transition Diagram

Combination Lock with Password 1010



IN	S[2]	S[1]	S[0]	OUT	S'[2]	S'[1]	S'[0]

Now we will start to build up to constructing a full mini computer processor that can perform 2 operations: addition and subtraction.

7 q3.py - Full 4-bit Arith Unit for ADD/SUB

Remember the 4-bit full adder you implemented in lec2.q7? We want to expand the logic of the 4-bit adder to be a full "arith" unit.

This unit should use the 4-bit adder to perform addition, if the ALU bit is 0. The unit should use the 4-bit adder to preform 2's-complement subtraction if the ALU bit is 1.

Eventually, we will also want to use this arith unit to perform comparison, for example. To do so, we will also compute 3 1-bit outputs, called "flag" bits.

- The "Z" flag should be 1 if your four-bit output is equal to 0000.
- The "N" flag should be 1 if your four-bit output is negative in 2's complement.
- The "V" flag should be 1 if overflow occured.

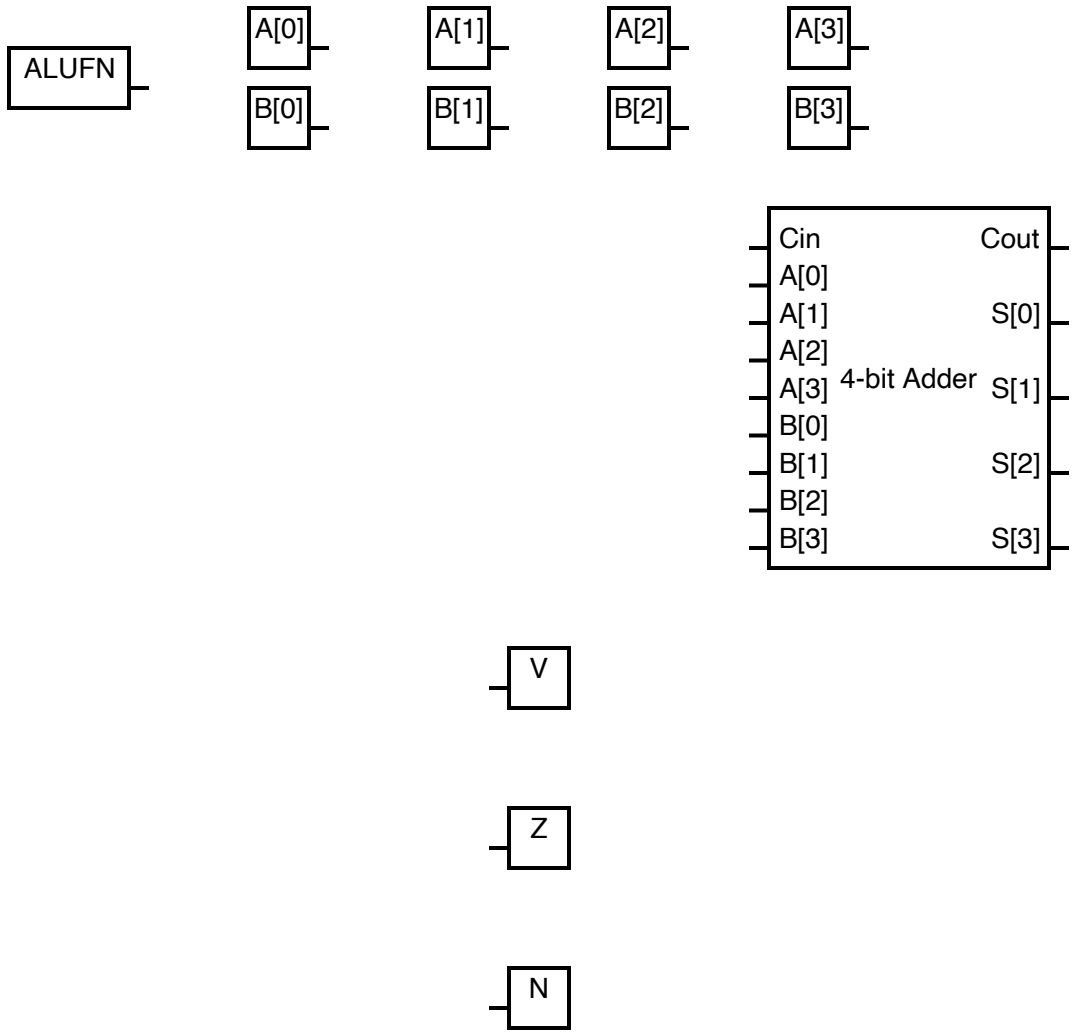
The logic for this is equivalent to:

$$V = XA_3 * XB_3 * \overline{S_3} + \overline{XA_3} * \overline{XB_3} * S_3$$

Consider, how can we use SUB and this Z bit to check if two values are equal? _____

Complete the implementation of the 4-bit arith unit below.

Complete the 4-Bit Arith Module by adding logic gates (AND,OR,NOT,XOR), muxes, and wires



8 q1.py - mini computer for N!

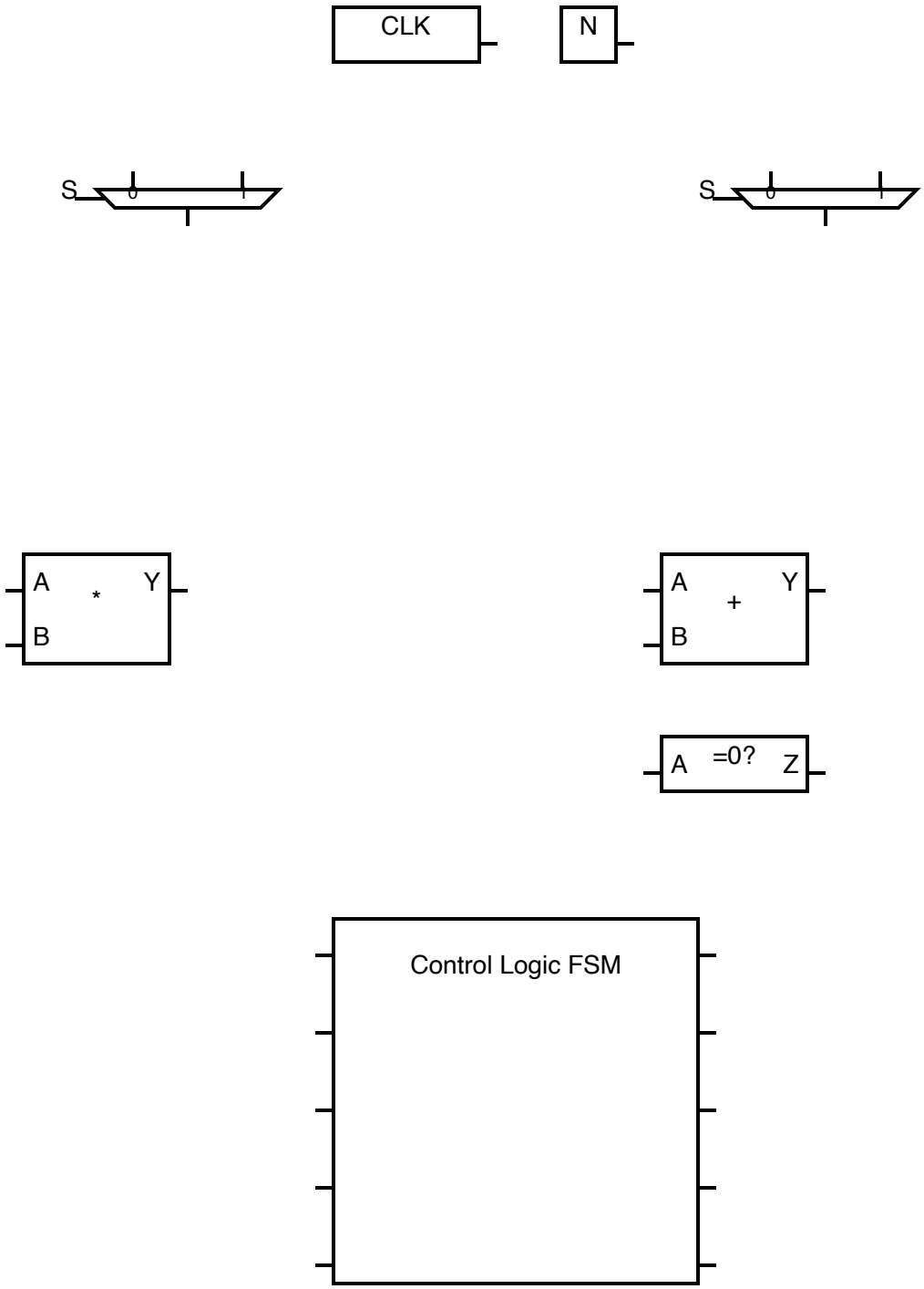
Let's create a mini computer that can compute $N!$.

To complete this task you will be creating two things:

- **The Data path**– wires, muxes, registers, etc. that combine to move the data of our computation from our two computation modules (multiplication and -1).
- **The Control path** – when should our computation stop? To do this, you can compute a control signal as input to our control logic module. This control logic module can implement a finite state machine and output any needed *control signals*, which our circuit can use as input.
  - Consider, that the registers' write enable signals should likely be output of our control logic.

Complete the circuit by adding wires, registers, and control logic description.

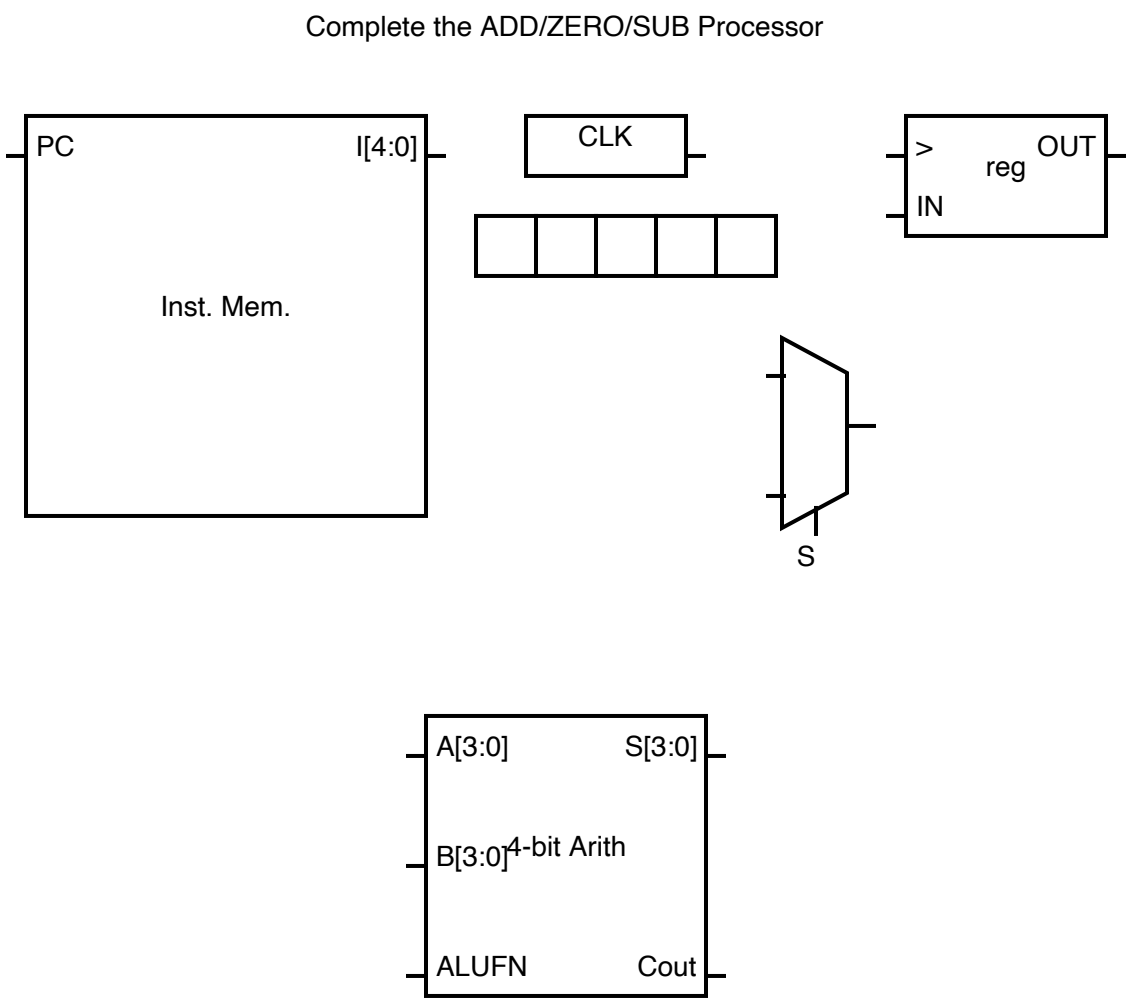
Complete the circuit so that it computes N! = N*(N-1)*(N-2)*...*3*2*1



9 q2.py - mini general purpose 4-bit computer w/ instruction memory? (zero/add)?

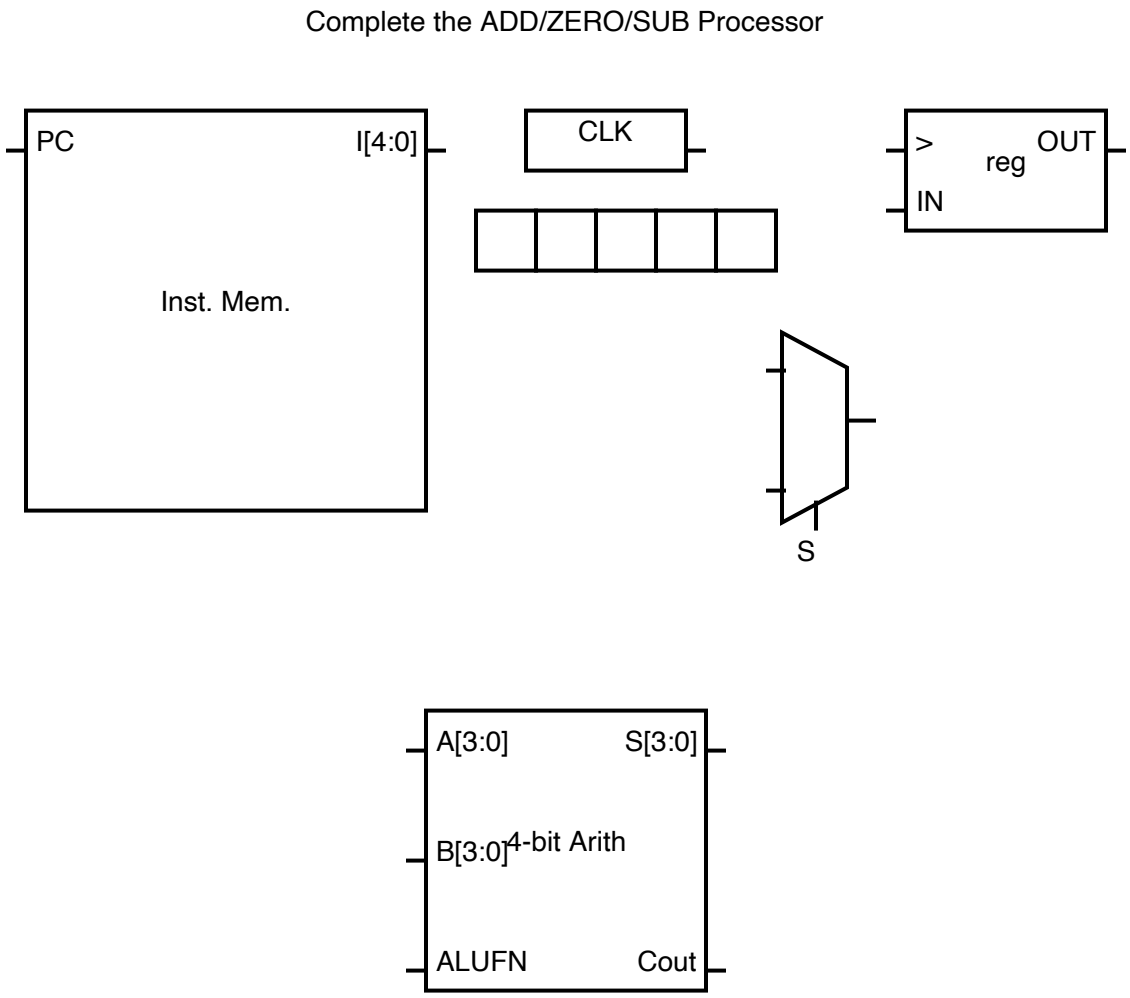
- Fetch, Decode, Execute
- 1 register
- Arith Unit
- Program Counter
- Instruction Memory
- CLK

```
ZERO
ADD 0
ADD 3
ADD -3
```



- Draw the wires to compelte the mini-computer
- Highlight the "control path", highlight the "data" path.
- What must CLK cycle be set to?

10 q4.py - add subtraction operation?

- 1 register
- Arith Unit w/ subtraction logic, new control signal for ALUFN
- Program Counter

```
ZERO
ADD 0
ADD 3
SUB 2
ADD -3
```